



MediAssist

Rapport Technique Détaillé

Application Mobile de Gestion Médicale

Développée avec Flutter & Google ML Kit

Package : `oz_medical` | Plateforme : Android

Préparées par : Zouhour Ameer – Ons Yangu

Enseignant : M. Sami Hadhri

Année universitaire : 2024–2025

Table des matières

1	Présentation du Projet	3
1.1	Vue d'ensemble	3
1.2	Informations générales	3
2	Services ML Kit Utilisés	4
2.1	Reconnaissance de texte (OCR)	4
2.2	Scan de codes-barres	4
2.3	Étiquetage d'images médicales	5
2.4	Identification et traduction de langues	5
2.5	Extraction d'entités	5
3	Choix Techniques	6
3.1	Framework et langage	6
3.2	Plugins principaux	6
3.2.1	Gestion d'état — Provider	6
3.2.2	Persistance des données	6
3.2.3	Interface utilisateur et UX	7
3.2.4	Services additionnels	7
4	Structure de l'Application	8
4.1	Architecture générale	8
5	Interfaces Utilisateur	9
5.1	Introduction	9
5.2	Flux d'authentification	9
5.2.1	Écran de connexion	9
5.2.2	Écran de création de compte	10
5.2.3	Réinitialisation du mot de passe	10
5.3	Tableau de bord principal	11
5.4	Gestion des médicaments	11
5.4.1	Liste des médicaments	11
5.5	Ordonnances médicales	12
5.6	Suivi des constantes vitales	13
5.7	Gestion des rendez-vous	13
5.7.1	Écran des consultations	13
5.8	Profil utilisateur	14

5.9	Récapitulatif des écrans	15
6	Système de Notifications	16
6.1	Architecture du NotificationService	16
6.2	Canaux de notification Android	16
6.3	Notifications de rappel médicament	16
6.4	Notifications de rendez-vous médical	17
7	Flux Fonctionnels	18
7.1	Flux de numérisation d'ordonnance (ML Kit OCR)	18
7.2	Flux de scan de code-barres médicament	18
7.3	Flux de gestion d'un rendez-vous	19
8	Sécurité et Qualité	20
8.1	Authentification	20
8.2	Traitement des données médicales	20
	Conclusion et Perspectives	21
	Conclusion	21
	Perspectives	21

Chapitre 1 — Présentation du Projet

1.1 Vue d'ensemble

MediAssist (identifiant de package : `oz_medical`) est une application mobile multiplateforme développée avec le framework **Flutter** (Dart). Elle a pour vocation d'assister les patients dans la gestion quotidienne de leur santé, en combinant un suivi médical structuré avec des capacités d'intelligence artificielle embarquées grâce à **Google ML Kit**.

Objectifs de l'application

- Numériser et analyser automatiquement les ordonnances médicales (OCR)
- Scanner les codes-barres des médicaments pour obtenir des informations détaillées
- Assurer le suivi des constantes vitales (glycémie, tension, fréquence cardiaque, poids, etc.)
- Gérer les médicaments avec rappels automatiques de prise
- Centraliser les consultations, prescriptions et dossiers médicaux
- Envoyer des notifications de rappel échelonnées pour les rendez-vous médicaux
- Proposer une interface multilingue (Français, Anglais, Arabe)

1.2 Informations générales

Paramètre	Valeur
Nom de l'application	MediAssist
Package identifier	<code>oz_medical</code>
Version	1.0.0
Langage	Dart (Flutter)
Plateformes cibles	Android
Langue par défaut	Français (<code>fr_FR</code>)
Langues supportées	Français, Anglais, Arabe
Base de données locale	Firebase Realtime Database
Service cloud	Firebase Firestore + Storage + Realtime DB

Table 1.1 – Informations générales du projet MediAssist

Chapitre 2 — Services ML Kit Utilisés

L'intelligence artificielle de l'application repose entièrement sur **Google ML Kit**, une suite de modèles d'apprentissage automatique *on-device*. La classe centrale **MLKitService** orchestre l'ensemble de ces capacités depuis `services/ml_kit_service.dart`.

2.1 Reconnaissance de texte (OCR)

Propriété	Détail
Plugin	<code>google_mlkit_text_recognition</code>
Classe Flutter	TextRecognizer
Script utilisé	Latin (<code>TextRecognitionScript.latin</code>)
Usage principal	Numérisation d'ordonnances médicales
Données extraites	Nom du médecin, patient, date, médicaments, dosages

Table 2.1 – Service OCR — Reconnaissance de texte

Le traitement d'une ordonnance suit le pipeline suivant : après chargement de l'image via `InputImage.fromFilePath()`, chaque `TextBlock` est parcouru ligne par ligne. Des expressions régulières détectent les motifs via `_isMedicationPattern()`, `_isDatePattern()`, `_isDoctorPattern()`, `_isPatientPattern()` et `_isDosagePattern()`. Le résultat est encapsulé dans un objet **PrescriptionScanResult**.

2.2 Scan de codes-barres

Propriété	Détail
Plugin	<code>google_mlkit_barcode_scanning</code>
Classe Flutter	BarcodeScanner
Usage principal	Identification de médicaments par code-barres
Données retournées	Code brut (<code>rawValue</code>), informations via MedicineInfo

Table 2.2 – Service de scan de codes-barres

Deux méthodes distinctes sont exposées : `scanBarcode()` retourne le code brut (`rawValue`), tandis que `scanMedicationBarcode()` appelle en plus `_fetchMedicineInfoFromCode()` pour retourner un objet **MedicineInfo** complet.

2.3 Étiquetage d'images médicales

Le plugin `google_mlkit_image_labeling` est utilisé via la classe `ImageLabeler` et la méthode `analyzeMedicalImage()` pour analyser des images médicales et retourner une liste d'étiquettes descriptives permettant d'identifier le contexte visuel d'une photo.

2.4 Identification et traduction de langues

- **Identification de langue** — Plugin `google_mlkit_language_id` (`LanguageIdentifier`) : détecte automatiquement la langue avec un seuil de confiance de 50 % (`confidenceThreshold: 0.5`).
- **Traduction on-device** — Plugin `google_mlkit_translation` (`OnDeviceTranslator`) : traduit les textes d'ordonnance entre le français, l'anglais et l'arabe sans requête réseau.

2.5 Extraction d'entités

Le plugin `google_mlkit_entity_extraction` avec la classe `EntityExtractor` (configurée avec `EntityExtractorLanguage.french`) identifie des entités structurées : dates, numéros de téléphone, adresses, et autres informations cliniques.

Chapitre 3 — Choix Techniques

3.1 Framework et langage

Flutter (version stable) avec le langage **Dart** a été choisi pour sa capacité à produire des applications natives performantes sur Android à partir d'une base de code unique. L'application utilise **Material Design 3** (`useMaterial3: true`) avec un thème clair/sombre dynamique géré par **ThemeProvider** et la locale active par **LanguageProvider**.

3.2 Plugins principaux

3.2.1 Gestion d'état — Provider

L'architecture repose sur le pattern **Provider** :

- **provider** — Injection de dépendances et gestion d'état global
- **AuthService** exposé via **ChangeNotifierProvider**
- **FirestoreService** mis à jour via **ProxyProvider<AuthService, FirestoreService>**
- **LanguageProvider** et **ThemeProvider** via **ChangeNotifierProvider**
- **SharedPreferences** et **RealtimeDbService** exposés via **Provider.value**

3.2.2 Persistance des données

L'application adopte une architecture de données cloud-first :

Plugin / Service	Rôle
<code>firebase_database</code>	Firebase Realtime DB (flux réactifs temps-réel)
<code>shared_preferences</code>	Préférences utilisateur (thème, langue, session)
FirestoreService	Synchronisation cloud des données médicales
StorageService	Stockage des images et documents
DataService	Streams réactifs (médicaments, consultations, vitaux)
<code>path_provider</code>	Résolution des chemins système (sauvegardes)

Table 3.1 – Plugins de persistance des données

3.2.3 Interface utilisateur et UX

Plugin	Rôle
<code>image_picker</code>	Sélection d'images (caméra ou galerie)
<code>flutter_local_notifications</code>	Rappels médicaments et rendez-vous (Android + iOS)
<code>timezone</code>	Fuseaux horaires pour notifications planifiées
<code>flutter_localizations</code>	Internationalisation (Material, Widgets, Cupertino)

Table 3.2 – Plugins d'interface et UX

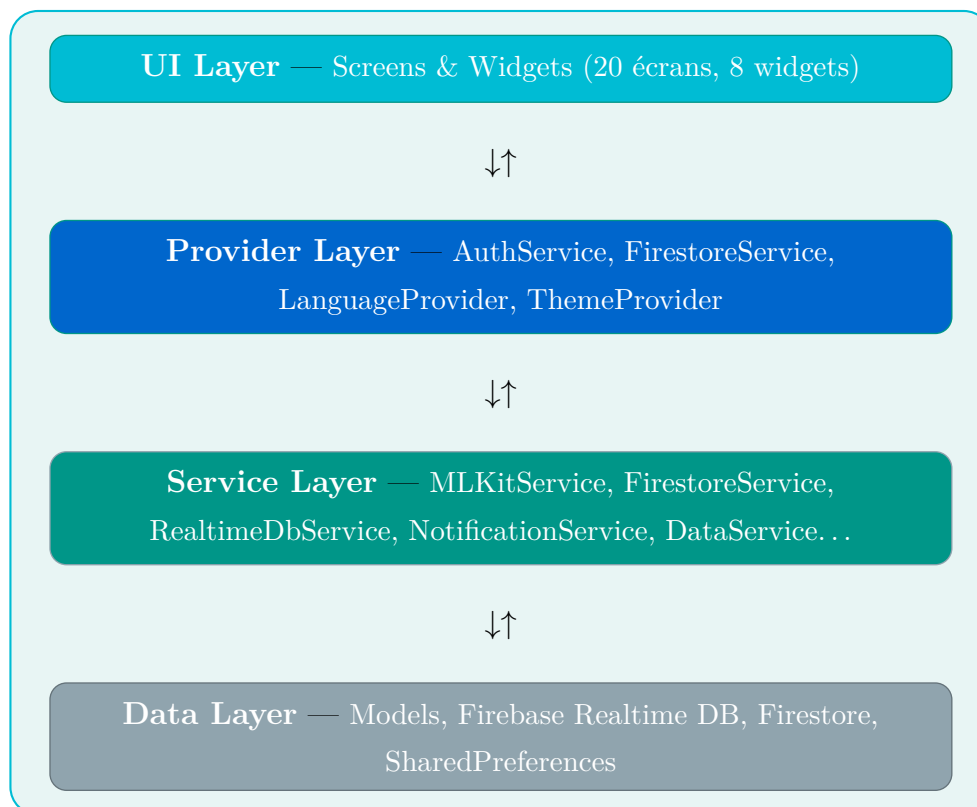
3.2.4 Services additionnels

- **BackupService** — Sauvegarde automatique JSON (intervalle : 7 jours, max 5 fichiers, via `share_plus`)
- **FeedbackService** — Retour haptique (vibrations) lors des interactions

Chapitre 4 — Structure de l'Application

4.1 Architecture générale

L'application suit une architecture en couches inspirée du pattern **MVVM**, adaptée aux conventions Flutter avec Provider :



Chapitre 5 — Interfaces Utilisateur

5.1 Introduction

L'interface utilisateur de MediAssist a été conçue selon les principes de **Material Design 3** pour offrir une expérience intuitive, cohérente et accessible. Les captures d'écran présentées dans ce chapitre illustrent l'ensemble des écrans principaux de l'application.

Charte graphique

- **Couleurs dominantes** : Teal médical (#00BCD4) et bleu professionnel (#0066CC)
- **Typographie** : Source Sans Pro (titres) et Roboto (corps)
- **Composants** : Cartes blanches avec ombres légères, coins arrondis
- **Iconographie** : Icônes Material Design cohérentes avec la signification médicale

5.2 Flux d'authentification

5.2.1 Écran de connexion

L'écran de connexion, illustré en figure 5.1, permet à l'utilisateur de s'identifier via son adresse email et son mot de passe. Il propose également un lien de récupération en cas de mot de passe oublié et une redirection vers la création de compte.



Figure 5.1 – Écran de connexion — *LoginScreen*

5.2.2 Écran de création de compte

La figure 5.2 présente l'écran d'inscription. L'utilisateur doit renseigner ses informations personnelles (prénom, nom, email) et choisir un mot de passe sécurisé. La confirmation du mot de passe est requise pour éviter les erreurs de saisie.

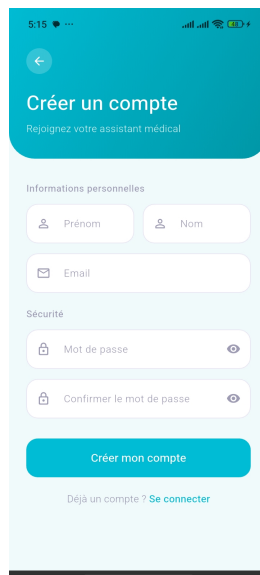


Figure 5.2 – Écran de création de compte — *RegisterScreen*

5.2.3 Réinitialisation du mot de passe

L'écran de réinitialisation (figure 5.3) permet à l'utilisateur de recevoir un lien de réinitialisation par email. Cette fonctionnalité utilise le service d'authentification Firebase.

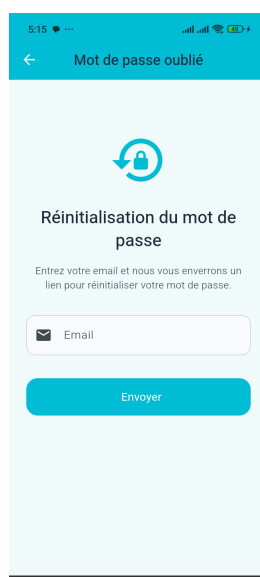


Figure 5.3 – Écran de réinitialisation — *ForgotPasswordScreen*

5.3 Tableau de bord principal

Une fois authentifié, l'utilisateur accède à l'écran d'accueil (figure 5.4). Cet écran sert de tableau de bord central avec :

- Un message de bienvenue personnalisé affichant le prénom de l'utilisateur
- Indicateurs récapitulatifs (médicaments, consultations, constantes, historique)
- Le prochain rendez-vous médical avec les détails (médecin, spécialité, date, heure)
- Une barre de navigation inférieure à 6 onglets pour accéder aux différentes sections

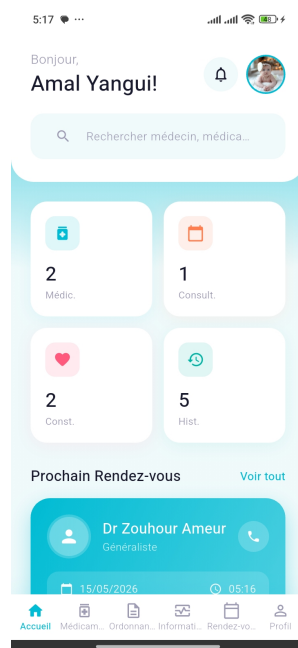


Figure 5.4 – Tableau de bord — *HomeScreen*

5.4 Gestion des médicaments

5.4.1 Liste des médicaments

L'écran *MedicationsScreen* (figure 5.5) présente la liste des traitements du patient avec :

- Une barre de recherche pour filtrer les médicaments
- Des filtres par statut : **Tout**, **En cours**, **Terminé**
- Pour chaque médicament : nom, dosage, horaire de prise, fréquence
- Un bouton **Marquer comme pris** pour enregistrer la prise
- Un bouton flottant d'ajout (FAB) avec icône scanner

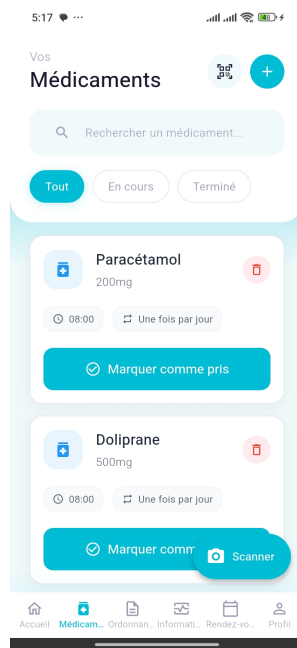


Figure 5.5 – Gestion des médicaments — *MedicationsScreen*

5.5 Ordonnances médicales

L'écran *PrescriptionsScreen* (figure 5.6) affiche l'historique des ordonnances numérisées. Chaque carte présente le nom du médecin prescripteur, la date de délivrance et le statut (Actif/Expiré). Les ordonnances peuvent être visualisées en détail ou partagées.

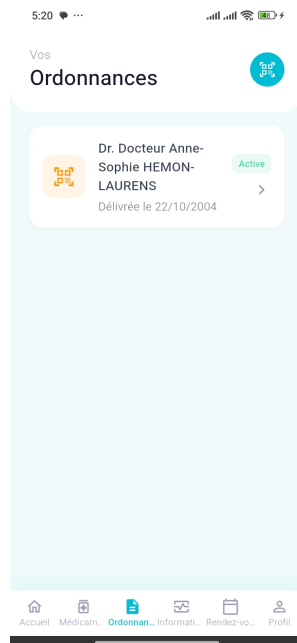


Figure 5.6 – Liste des ordonnances — *PrescriptionsScreen*

5.6 Suivi des constantes vitales

L'écran *VitalSignsScreen* (figure 5.7) permet le suivi des paramètres de santé :

- Vue d'ensemble des constantes disponibles (glycémie, tension artérielle)
- Graphique d'évolution temporelle avec moyenne calculée
- Liste des mesures récentes avec détails et alerte en cas d'anomalie
- Indicateur coloré pour les valeurs hors normes (exemple : **Hypoglycémie** pour 40.0 mg/dL)



Figure 5.7 – Suivi des constantes — *VitalSignsScreen*

5.7 Gestion des rendez-vous

5.7.1 Écran des consultations

L'écran *ConsultationsScreen* (figure 5.8) est organisé en deux onglets :

- **À venir** : Liste des consultations futures avec médecin, spécialité, date/heure et type (présentiel)
- **Historique** : Consultations passées avec compte-rendu
- Pour chaque consultation, un bouton **Préparer ma consultation** permet d'accéder à l'écran de préparation



Figure 5.8 – Gestion des rendez-vous — *ConsultationsScreen*

5.8 Profil utilisateur

L'écran *ProfileScreen* (figure 5.9) centralise les informations personnelles et médicales du patient :

- **En-tête** : Photo de profil, nom complet, email, âge
- **Informations personnelles** : Téléphone, date de naissance, adresse
- **Données médicales** : Groupe sanguin, poids, etc.
- Chaque champ est modifiable via l'icône d'édition

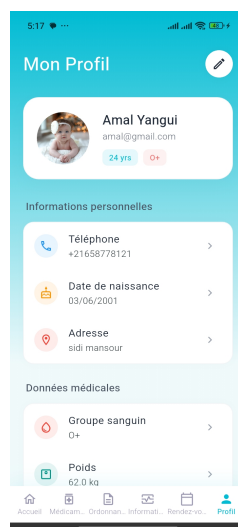


Figure 5.9 – Profil patient — *ProfileScreen*

5.9 Récapitulatif des écrans

Écran	Fonctionnalités
<i>LoginScreen</i>	Authentification email/mot de passe
<i>RegisterScreen</i>	Inscription avec profil médical complet
<i>ForgotPasswordScreen</i>	Réinitialisation de mot de passe
<i>HomeScreen</i>	Tableau de bord, 6 onglets, accès rapides ML Kit
<i>MedicationsScreen</i>	Liste médicaments, filtres, FAB scan
<i>AddMedicationScreen</i>	Ajout/édition d'un médicament
<i>PrescriptionsScreen</i>	Liste des ordonnances
<i>VitalSignsScreen</i>	Graphiques de constantes, alertes anomalies
<i>AddVitalScreen</i>	Ajout/édition d'une mesure
<i>ScanPrescriptionScreen</i>	OCR ordonnance + traduction multilingue
<i>ScanBarcodeScreen</i>	Identification médicament par code-barres
<i>AnalyzeImageScreen</i>	Étiquetage d'images médicales
<i>ConsultationsScreen</i>	Agenda médical avec filtrage
<i>AddConsultationScreen</i>	Ajout/édition d'une consultation
<i>PrepareConsultationScreen</i>	Préparation de consultation (symptômes, questions)
<i>HistoryScreen</i>	Historique médical
<i>HospitalsDoctorsScreen</i>	Localisation hôpitaux et médecins
<i>ProfileScreen</i>	Dossier patient complet
<i>EditProfileScreen</i>	Édition du profil patient
<i>SettingsScreen</i>	Thème, langue, notifications, sauvegarde

Table 5.1 – Récapitulatif des 20 écrans de l'application

Chapitre 6 — Système de Notifications

6.1 Architecture du NotificationService

Le **NotificationService** est implémenté en **singleton** via le pattern factory. L'initialisation est effectuée de manière asynchrone dans `main.dart` après `runApp()`, afin de ne pas bloquer le rendu initial de l'interface.

6.2 Canaux de notification Android

Identifiant	Nom	Usage
<code>medication_channe</code>	Rappels de médicaments	Rappels quotidiens de prise
<code>appointment_chann</code>	Rappels de rendez-vous	Rappels échelonnés consultations
<code>vital_channel</code>	Rappels de constantes	Rappels de mesure des signes vitaux
<code>general_channel</code>	Notifications générales	Notifications immédiates

Table 6.1 – Canaux de notification Android

6.3 Notifications de rappel médicament

La méthode `scheduleMedicationReminder(Medication medication)` planifie une notification par horaire de prise défini dans `medication.times`. Les rappels sont configurés avec les caractéristiques suivantes :

Propriétés des rappels médicament

- **Répétition** : Quotidienne à heure fixe (`DateTimeComponents.time`)
- **Mode** : `exactAllowWhileIdle` (déclenché même en veille système)
- **Priorité** : `Importance.max` / `Priority.high`
- **Payload** : `'medication_{id}'` (navigation à la réception)
- **Unicité** : Les rappels existants sont annulés avant replanification

6.4 Notifications de rendez-vous médical

La méthode `scheduleConsultationReminder(Consultation consultation)` planifie automatiquement **4 rappels échelonnés** avant chaque consultation. Seuls les rappels dont l'heure calculée est dans le futur sont enregistrés.

Délai avant RDV	ID (suffixe)	Message de notification
J-1 (24 heures)	hashCode + 1	« Vous avez une consultation avec {médecin} demain à {HH :mm} »
H-2 (2 heures)	hashCode + 2	« Consultation avec {médecin} dans 2 heures »
30 minutes	hashCode + 3	« Votre consultation avec {médecin} commence dans 30 minutes »
10 minutes	hashCode + 4	« Consultation imminente avec {médecin} (dans 10 min) »

Table 6.2 – Rappels échelonnés pour les rendez-vous médicaux

Chapitre 7 — Flux Fonctionnels

7.1 Flux de numérisation d'ordonnance (ML Kit OCR)

Étapes du flux de scan d'ordonnance

1. L'utilisateur accède à *ScanPrescriptionScreen*
2. Sélection de la source : **Caméra** ou **Galerie** (via **ImagePicker**)
3. Le retour haptique confirme l'action (**FeedbackService**)
4. **MLKitService.scanPrescription()** traite l'image :
 - Conversion en **InputImage** via `InputImage.fromFilePath()`
 - Reconnaissance de texte par **TextRecognizer** (script latin)
 - Extraction par regex : médicaments, dates, médecin, patient
 - Détection automatique de la langue (**LanguageIdentifier**)
5. Affichage du résultat **PrescriptionScanResult**
6. Option de traduction vers fr/en/ar (**OnDeviceTranslator**)
7. Sauvegarde des médicaments extraits (**FirestoreService**)

7.2 Flux de scan de code-barres médicament

Étapes du scan code-barres

1. Accès à *ScanBarcodeScreen*
2. Capture photo du médicament (caméra ou galerie via **ImagePicker**)
3. **MLKitService.scanBarcode()** retourne le code brut (`rawValue`)
4. **MLKitService.scanMedicationBarcode()** appelle
`_fetchMedicineInfoFromCode()`
5. Affichage des informations complètes **MedicineInfo**

7.3 Flux de gestion d'un rendez-vous

Cycle de vie d'une consultation

1. Création via *AddConsultationScreen* (médecin, date, type, lieu, symptômes)
2. Planification automatique des 4 notifications par **NotificationService**
3. Préparation via *PrepareConsultationScreen*
4. Réception des rappels : J-1, H-2, 30 min, 10 min
5. Mise à jour du statut post-consultation (completed / cancelled)
6. Saisie du diagnostic, notes et ordonnances issues

Chapitre 8 — Sécurité et Qualité

8.1 Authentification

AuthService (ChangeNotifier) centralise la gestion de session. L'état de connexion est persisté via `shared_preferences` (`user_logged_in`, `user_id`, `user_email`, `user_name`). La navigation initiale est pilotée par un `Consumer<AuthService>` : si `authService.isLoading` est vrai, un indicateur de chargement est affiché ; si `authService.isLoggedIn` est vrai, l'utilisateur est redirigé vers *HomeScreen*.

8.2 Traitement des données médicales

- Toutes les opérations ML Kit s'exécutent **on-device** : les données médicales ne quittent jamais l'appareil lors du traitement IA
- Taille maximale des images : **5 Mo** (`AppConstants.maxImageSizeMB`)
- Historique des constantes vitales plafonné à **100 entrées** par type
- Liste des médicaments récents limitée à **10 entrées**
- Sauvegardes conservées sur **5 fichiers** maximum

Conclusion et Perspectives

Conclusion

MediAssist (`oz_medical`) est une application médicale mobile complète qui tire parti de la puissance de **Google ML Kit** pour offrir des fonctionnalités d'intelligence artificielle directement sur l'appareil, sans compromettre la confidentialité des données de santé.

L'architecture en couches (Provider + Services + Modèles), combinée à une persistance cloud réactive (Firebase Realtime Database + Firestore), garantit à la fois la résilience et la synchronisation multi-appareils. Le système de notifications à 4 niveaux de rappel assure une gestion proactive du suivi médical du patient.

Au terme de ce développement, les objectifs initiaux sont atteints :

- **6 services Google ML Kit** intégrés et opérationnels *on-device*
- **20 écrans** couvrant l'ensemble du parcours patient
- **Système de notifications** à 4 niveaux de rappel pour les rendez-vous médicaux
- **Architecture robuste** en couches avec gestion d'état réactive
- **Persistance cloud** garantissant la disponibilité et la synchronisation
- **Multilingue** (français, anglais, arabe) avec traduction automatique *on-device*

Perspectives

1. **Intégration de modèles ML Kit personnalisés** pour la reconnaissance des ordonnances arabes.
2. **Connexion aux objets connectés** et capteurs biométriques (Bluetooth Low Energy).
3. **Détection des interactions médicamenteuses** via une base de données pharmacologique.
4. **Module de téléconsultation** intégré (vidéo en temps réel).
5. **Renforcement de la sécurité** et conformité réglementaire (RGPD / normes médicales).

6. Extension vers le web et le desktop (Flutter Web / Desktop).**Feuille de route — MediAssist**

Court terme	Moyen terme	Long terme
OCR arabe (ML Kit)	Objets connectés BLE	Téléconsultation
Chiffrement données	Interactions médicaments	Certification DML
Conformité RGPD	Modèles ML custom	Version web/desktop
Tests unitaires	Tableau analytique	Partenariats cliniques